

C++ 프로그래밍 (다형성)

8주차

목차

- 다형성
- 가상 함수
- 순수 가상 함수

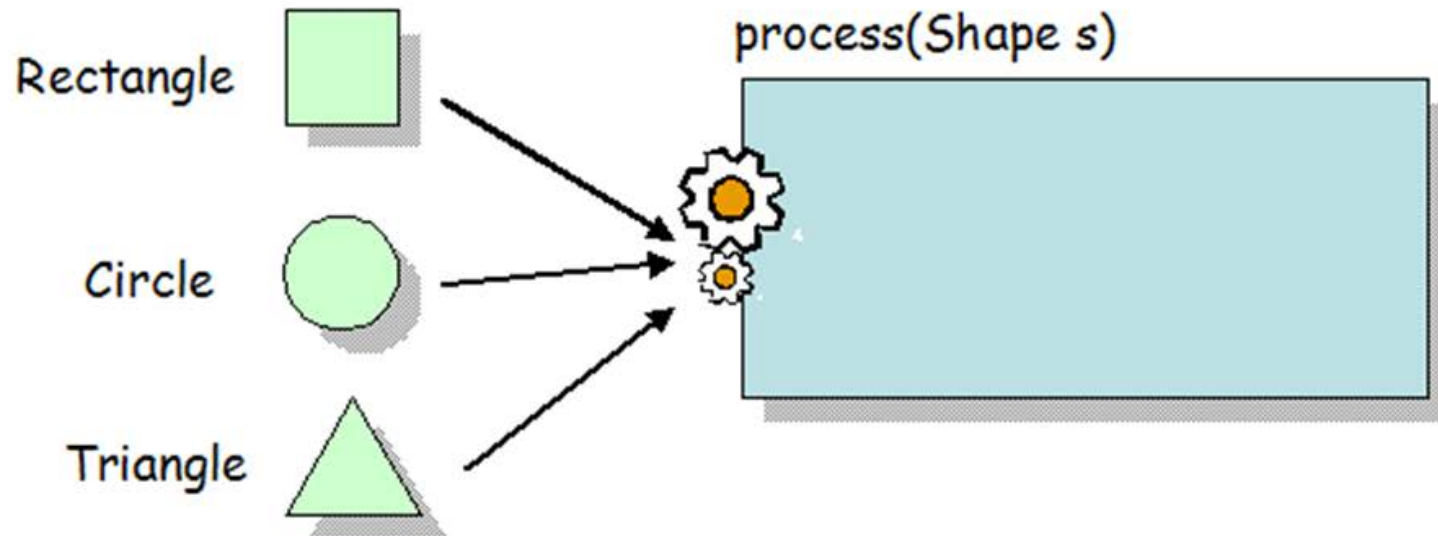
다형성

- 다형성이란?
 - 객체들의 타입이 다르면 똑같은 메시지가 전달되더라도 서로 다른 동작을 하는 것
 - 객체 지향 기법에서 하나의 코드로 다양한 타입의 객체를 처리하는 중요한 기술



다형성 (계속)

- 다형성 이란? (계속)



다형성은 다양한 객체들을 하나의 코드로 처리하는 기술입니다.



다형성 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Shape {                      // 일반적인도형을 나타내는 부모 클래스
protected:
    int x, y;

public:
    void draw() {
        cout << "Shape Draw" << endl;
    }
    void setOrigin(int x, int y){
        this->x = x;
        this->y = y;
    }
};
```

다형성 (계속)

- 예제 (계속)

```
class Rectangle : public Shape {  
private:  
    int width, height;  
public:  
    void setWidth(int w) {  
        width = w;  
    }  
    void setHeight(int h) {  
        height = h;  
    }  
    void draw() {  
        cout << "Rectangle Draw"<< endl;  
    }  
};
```

다형성 (계속)

- 객체 포인터의 형 변환

객체 포인터의 형변환

상향 형변환(**upcasting**):
자식 클래스 타입을 부모 클래스타입으로 변환

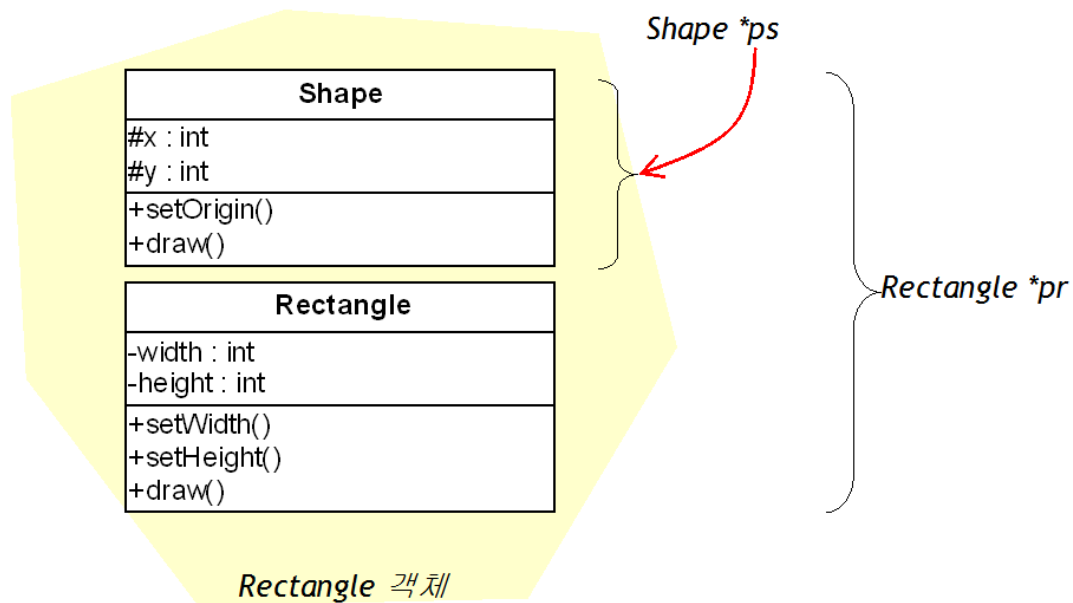
하향 형변환(**downcasting**):
부모 클래스 타입을 자식 클래스타입으로 변환

다형성 (계속)

- 상향 형변환

- Shape *ps = new Rectangle(); ~~// OK!~~
- ps->setOrigin(10, 10); // OK!

상향 형변환



Rectangle 객체를 Shape 포인터로 가리키면 Shape에 정의된 부분밖에 가리키지 못한다.

다형성 (계속)

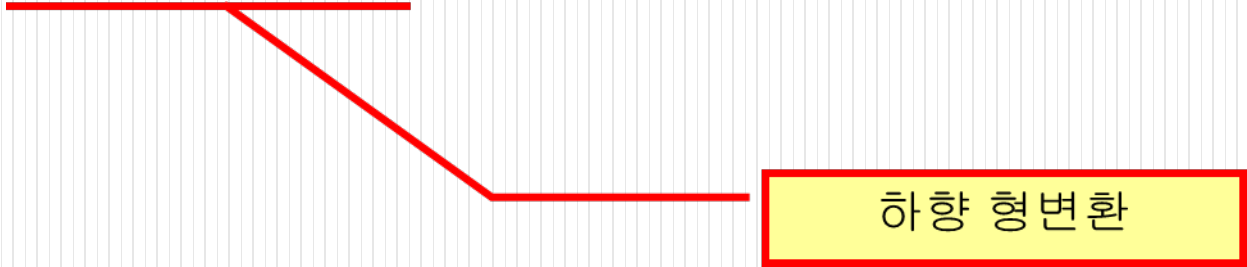
- 하향 형변환

- Shape *ps = new Rectangle();

- 여기서 ps를 통하여 Rectangle의 멤버에 접근하려면?

- Rectangle *pr = (Rectangle *) ps;
pr->setWidth(100);

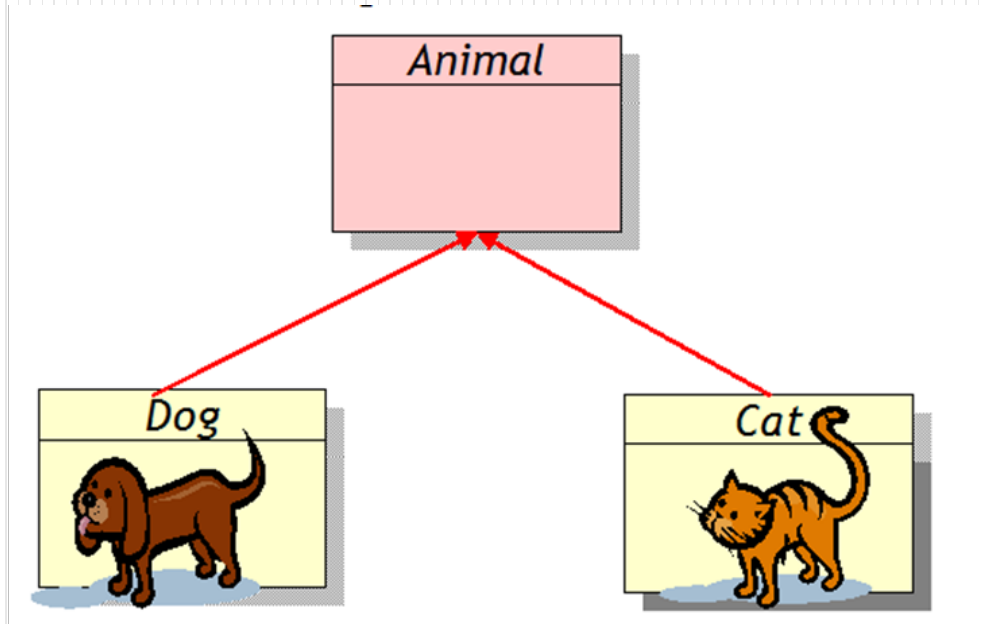
- ((Rectangle *) ps)->setWidth(100);



하향 형변환

다형성 (계속)

- 상속과 객체 포인터



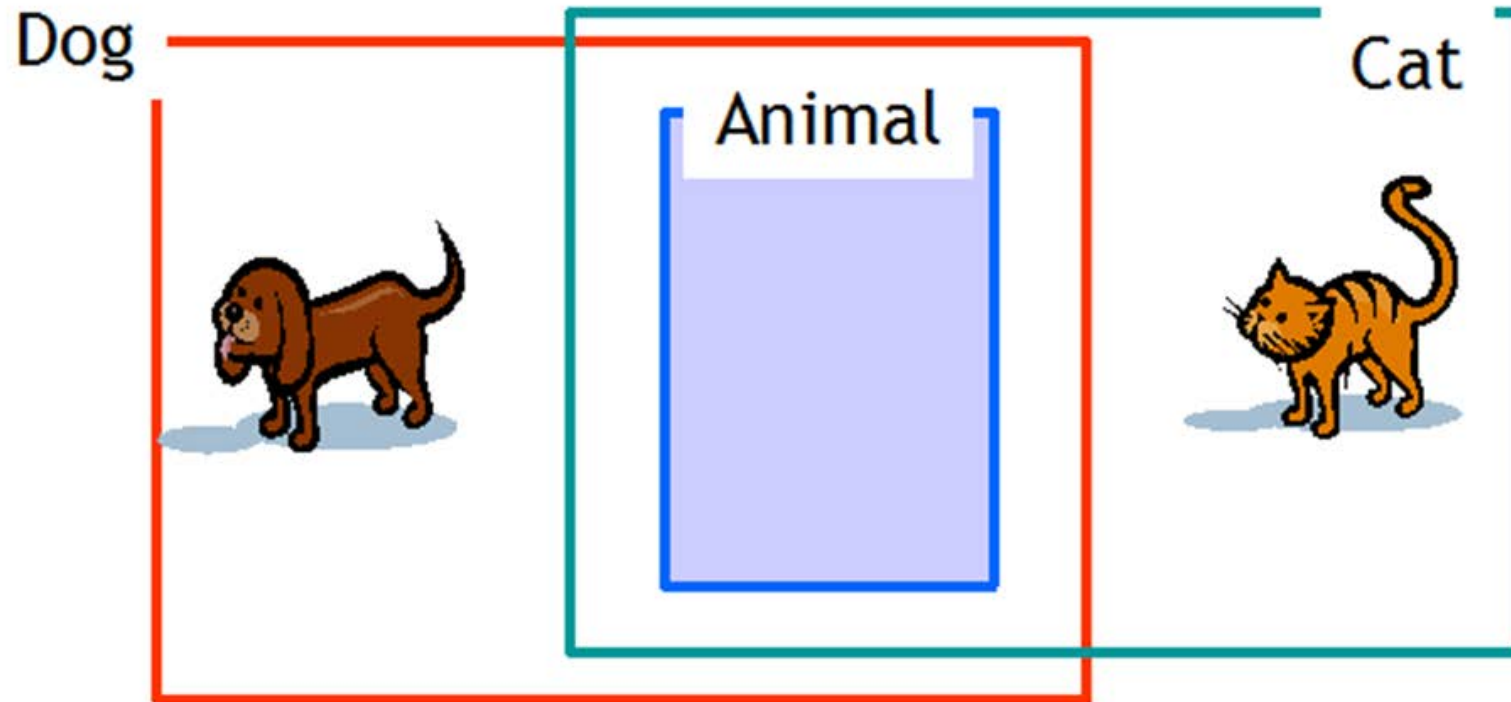
`Animal *pa = new Dog(); // OK!`

Animal 타입
포인터로 Dog
객체를 참조하니
틀린 거 같지만
올바른 문장!!



다형성 (계속)

- 상속과 객체 포인터 (계속)
 - 자식 클래스 객체는 부모 클래스 객체를 포함하고 있기 때문



다형성 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Shape {                      // 일반적인도형을 나타내는 부모 클래스
protected:
    int x, y;

public:
    void draw() {
        cout << "Shape Draw" << endl;
    }
    void setOrigin(int x, int y){
        this->x = x;
        this->y = y;
    }
};
```

다형성 (계속)

- 예제 (계속)

```
class Rectangle : public Shape {  
private:  
    int width, height;  
public:  
    void setWidth(int w) {  
        width = w;  
    }  
    void setHeight(int h) {  
        height = h;  
    }  
    void draw() {  
        cout << "Rectangle Draw"<< endl;  
    }  
};
```

다형성 (계속)

- 예제 (계속)

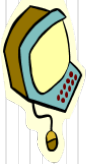
```
class Circle : public Shape {  
private:  
    int radius;  
  
public:  
    void setRadius(int r) {  
        radius = r;  
    }  
    void draw() {  
        cout << "Circle Draw"<< endl;  
    }  
};
```

다형성 (계속)

- 예제 (계속)

```
int main()
{
    Shape *ps = new Rectangle();           // OK!

    ps->setOrigin(10, 10);
    ps->draw();
    ((Rectangle *)ps)->setWidth(100);      // Rectangle의 setWidth() 호출
    delete ps;
}
```



Shape Draw

계속하려면 아무 키나 누르십시오 . . .

다형성 (계속)

- 함수의 매개 변수
 - 자식 클래스보다는 부모 클래스 타입으로 선언 하는 것이 좋다.

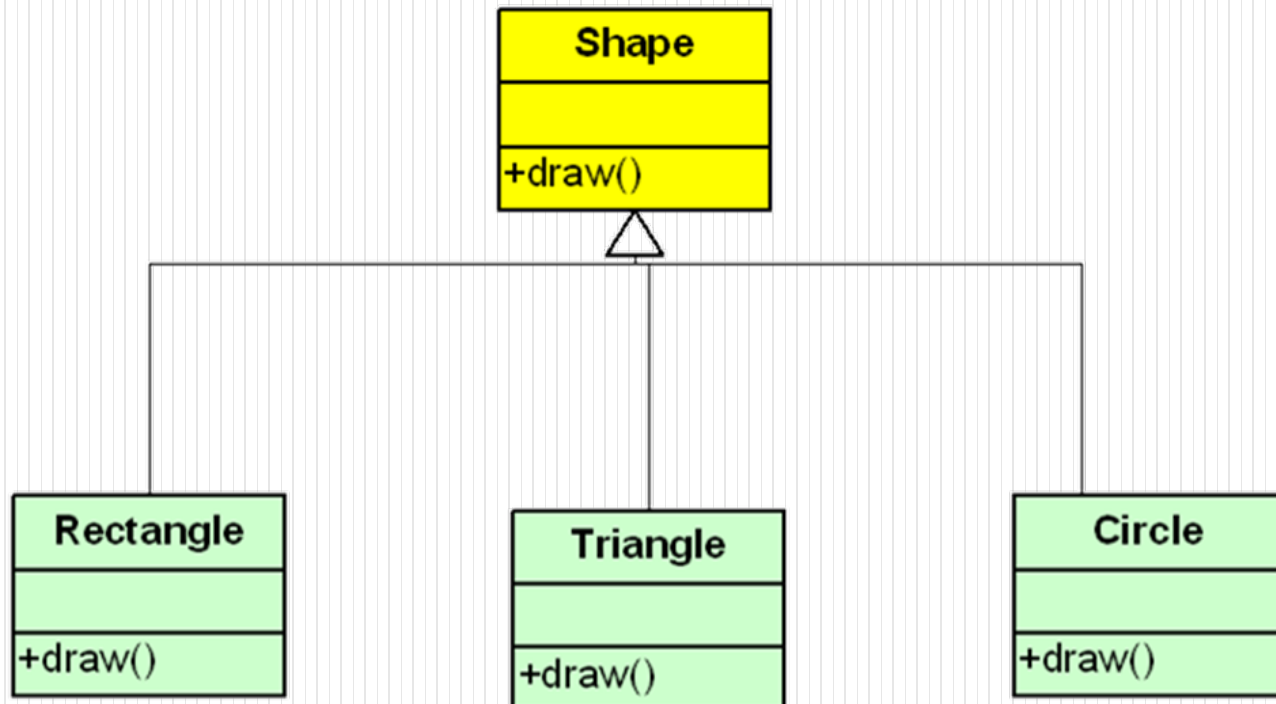
```
void move(Shape& s, int sx, int sy)
{
    s.setOrigin(sx, sy);
}
int main()
{
    Rectangle r;
    move(r, 0, 0);

    Circle c;
    move(c, 10, 10);
    return 0;
}
```

모든 도형을 받을 수 있다.

가상 함수

- 가상함수
 - 자식 클래스 객체를 부모 클래스 객체로 취급하는 것이 어디에 쓸모 있을까?
 - 상속 계층도를 가정하면.....



가상 함수 (계속)

- 예제

```
class Shape {  
    ...  
}  
class Rectangle : public Shape {  
    ...  
}  
int main()  
{  
    Shape *ps = new Rectangle(); // OK!  
    ps->draw();                  // 어떤 draw()가 호출되는가?  
}
```

Shape
포인터이기
때문에
Shape의
draw()가 호출

Shape Draw

가상 함수 (계속)

- 가상 함수
 - 만약 Shape포인터를 통하여 멤버 함수를 호출하더라도 도형의 종류에 따라서 서로 다른 draw()가 호출된다면 상당히 유용할 것이다.
 - 즉 사각형인 경우에는 사각형을 그리는 draw()가 호출되고 원의 경우에는 원을 그리는 draw()가 호출된다면 좋을 것이다.

가상 함수 (계속)

- 예제

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Shape {
protected:
    int x, y;
```

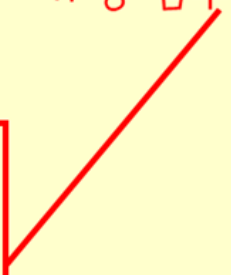
```
public:
```

```
    void setOrigin(int x, int y){
        this->x = x;
        this->y = y;
    }
```

```
    virtual void draw() {
        cout << "Shape Draw" << endl;
    }
```

```
};
```

가상 함수 정의



가상 함수 (계속)

- 예제 (계속)

```
class Rectangle : public Shape {  
private:
```

```
    int width, height;
```

```
public:
```

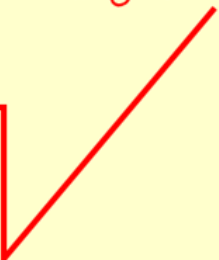
```
    void setWidth(int w) {  
        width = w;  
    }
```

```
    void setHeight(int h) {  
        height = h;  
    }
```

```
    void draw() {  
        cout << "Rectangle Draw" << endl;  
    }
```

```
};
```

재정의



가상 함수 (계속)

- 예제 (계속)

```
class Circle : public Shape {  
private:
```

```
    int radius;
```

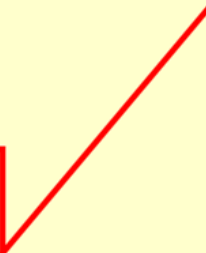
```
public:
```

```
    void setRadius(int r) {  
        radius = r;  
    }
```

```
    void draw() {  
        cout << "Circle Draw"<< endl;  
    }
```

```
};
```

재정의



가상 함수 (계속)

- 예제 (계속)

```
int main()
{
    Shape *ps = new Rectangle(); // OK!
    ps->draw();
    delete ps;

    Shape *ps1 = new Circle();      // OK!
    ps1->draw();
    delete ps1;
    return 0;
}
```

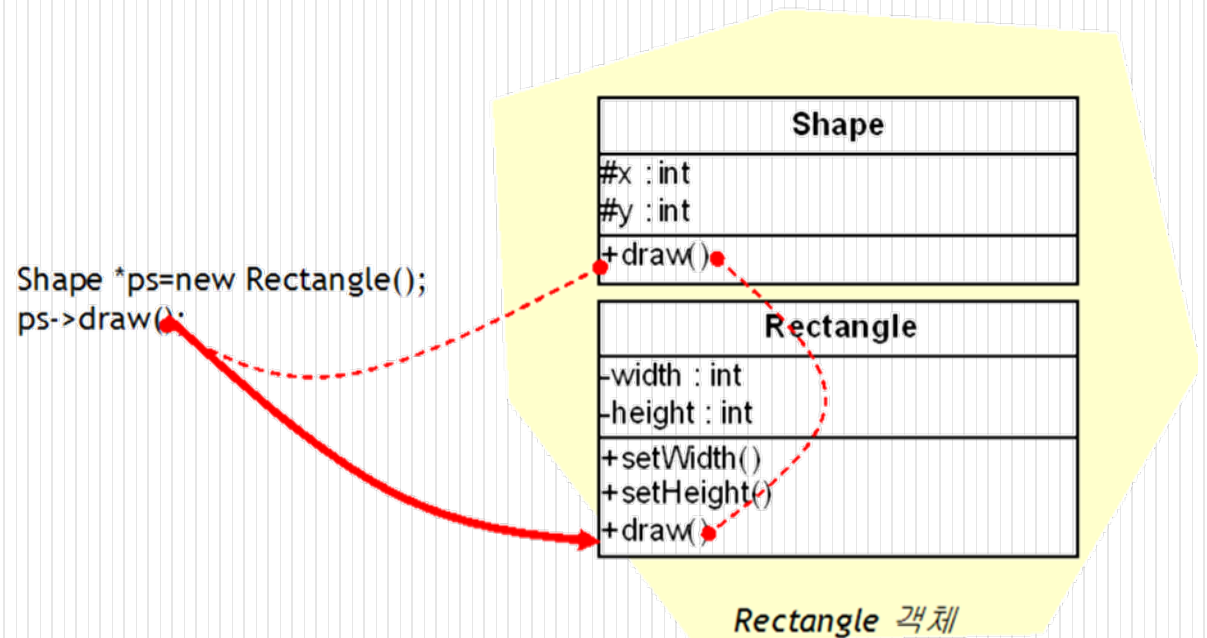
Rectangle Draw

Circle Draw

계속하려면 아무 키나 누르십시오 . . .

가상 함수 (계속)

- 동적 바인딩
 - 컴파일 단계에서 모든 바인딩이 완료되는 것을 정적 바인딩(static binding)이라고 한다.
 - 반대로 바인딩이 실행 시까지 연기되고 실행 시간에 실제 호출되는 함수를 결정하는 것을 동적 바인딩(dynamic binding), 또는 지연 바인딩(late binding)이라고 한다.



가상 함수 (계속)

- 정적 바인딩과 동적 바인딩

바인딩의 종류	특징	속도	대상
정적 바인딩 (dynamic binding)	컴파일 시간에 호출 함수가 결정된다.	빠르다	일반 함수
동적 바인딩 (static binding)	실행 시간에 호출 함수가 결정된다.	느다	가상 함수

가상 함수 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Shape {
protected:
    int x, y;

public:
    virtual void draw() {
        cout << "Shape Draw";
    }
    void setOrigin(int x, int y){
        this->x = x;
        this->y = y;
    }
};
```

가상 함수 (계속)

- 예제 (계속)

```
class Rectangle : public Shape {  
private:  
    int width, height;  
  
public:  
    void setWidth(int w) {  
        width = w;  
    }  
  
    void setHeight(int h) {  
        height = h;  
    }  
  
    void draw() {  
        cout << "Rectangle Draw" << endl;  
    }  
};
```

가상 함수 (계속)

- 예제 (계속)

```
class Circle : public Shape {  
private:  
    int radius;  
  
public:  
    void setRadius(int r) {  
        radius = r;  
    }  
    void draw() {  
        cout << "Circle Draw" << endl;  
    }  
};
```

가상 함수 (계속)

- 예제 (계속)

```
class Triangle: public Shape {
private:
    int base, height;
public:
    void draw() {
        cout << "Triangle Draw" << endl;
    }
};

int main()
{
    Shape *arrayOfShapes[3];

    arrayOfShapes[0] = new Rectangle();
    arrayOfShapes[1] = new Triangle();
    arrayOfShapes[2] = new Circle();
    for (int i = 0; i < 3; i++) {
        arrayOfShapes[i]->draw();
    }
}
```

```
Rectangle Draw
Triangle Draw
Circle Draw
```

가상 함수 (계속)

- 참조자와 가상함수
 - 참조자인 경우에는 다형성이 동작될 것인가?
 - > 참조자도 포인터와 마찬가지로 모든 것이 동일하게 적용

가상 함수 (계속)

- 예제

```
#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void speak() { cout <<"Animal speak()" << endl; }
};

class Dog : public Animal
{
public:
    void speak() { cout <<"멍멍" << endl; }
};

class Cat : public Animal
{
public:
    void speak() { cout <<"야옹" << endl; }
};
```

가상 함수 (계속)

- 예제 (계속)

```
int main()
{
    Dog d;
    Animal &a1 = d;
    a1.speak();

    Cat c;
    Animal &a2 = c;
    a2.speak();
    return 0;
}
```

멍멍
야옹

가상 함수 (계속)

- 가상 소멸자
 - 다형성을 사용하는 과정에서 소멸자를 virtual로 해주지 않으면 문제가 발생한다.

순수 가상 함수

- 순수 가상 함수
 - 함수 헤더만 존재하고 함수의 몸체는 없는 함수

```
virtual 반환형 함수이름(매개변수 리스트) = 0;
```

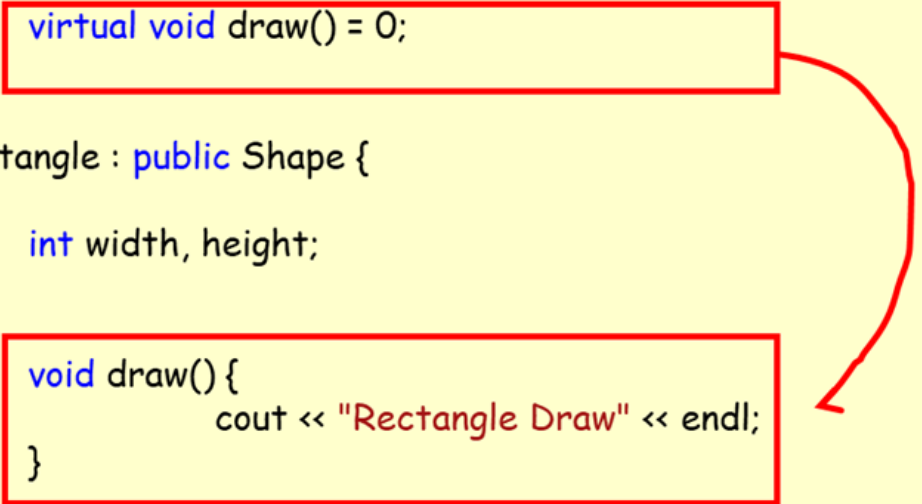
예) virtual void draw() = 0;

- 추가 클래스(abstract class)는 순수 가상함수를 하나라도 가지고 있는 클래스

순수 가상 함수 (계속)

- 예제

```
class Shape {  
protected:  
    int x, y;  
  
public:  
    ...  
    virtual void draw() = 0;  
};  
  
class Rectangle : public Shape {  
private:  
    int width, height;  
  
public:  
    void draw() {  
        cout << "Rectangle Draw" << endl;  
    }  
};
```



순수 가상 함수 (계속)

- 예제 (계속)

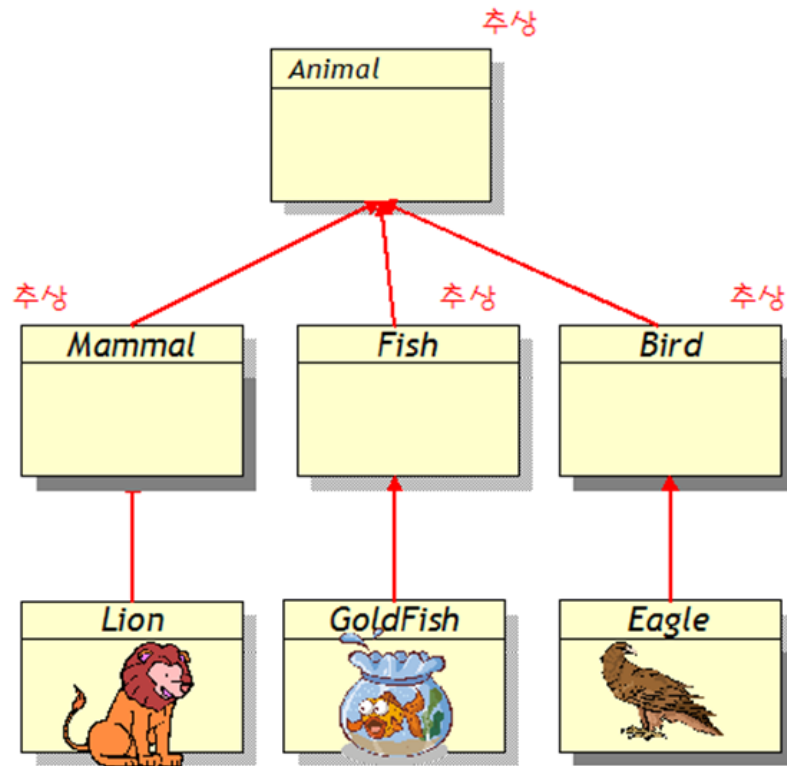
```
int main()
{
    Shape *ps = new Rectangle();    // OK!
    ps->draw();                      // Rectangle의 draw()가 호출된다.
    delete ps;

    return 0;
}
```

Rectangle Draw

순수 가상 함수 (계속)

- 추상 클래스
 - 순수 가상 함수를 가지고 있는 클래스
 - 추상적인 개념을 표현하는데 적당함.



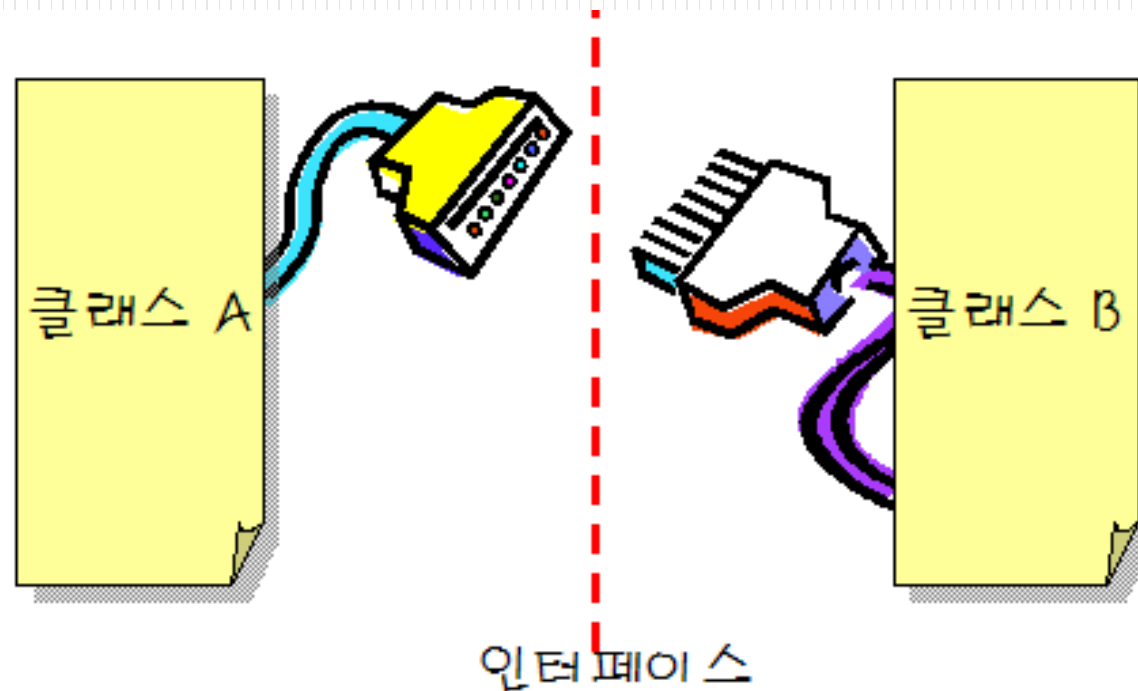
순수 가상 함수 (계속)

- 예제

```
class Animal {  
    virtual void move() = 0;  
    virtual void eat() = 0;  
    virtual void speak() = 0;  
};  
  
class Lion : public Animal {  
    void move(){  
        cout << "사자의 move() << endl;  
    }  
    void eat(){  
        cout << "사자의 eat() << endl;  
    }  
    void speak(){  
        cout << "사자의 speak() << endl;  
    }  
};
```

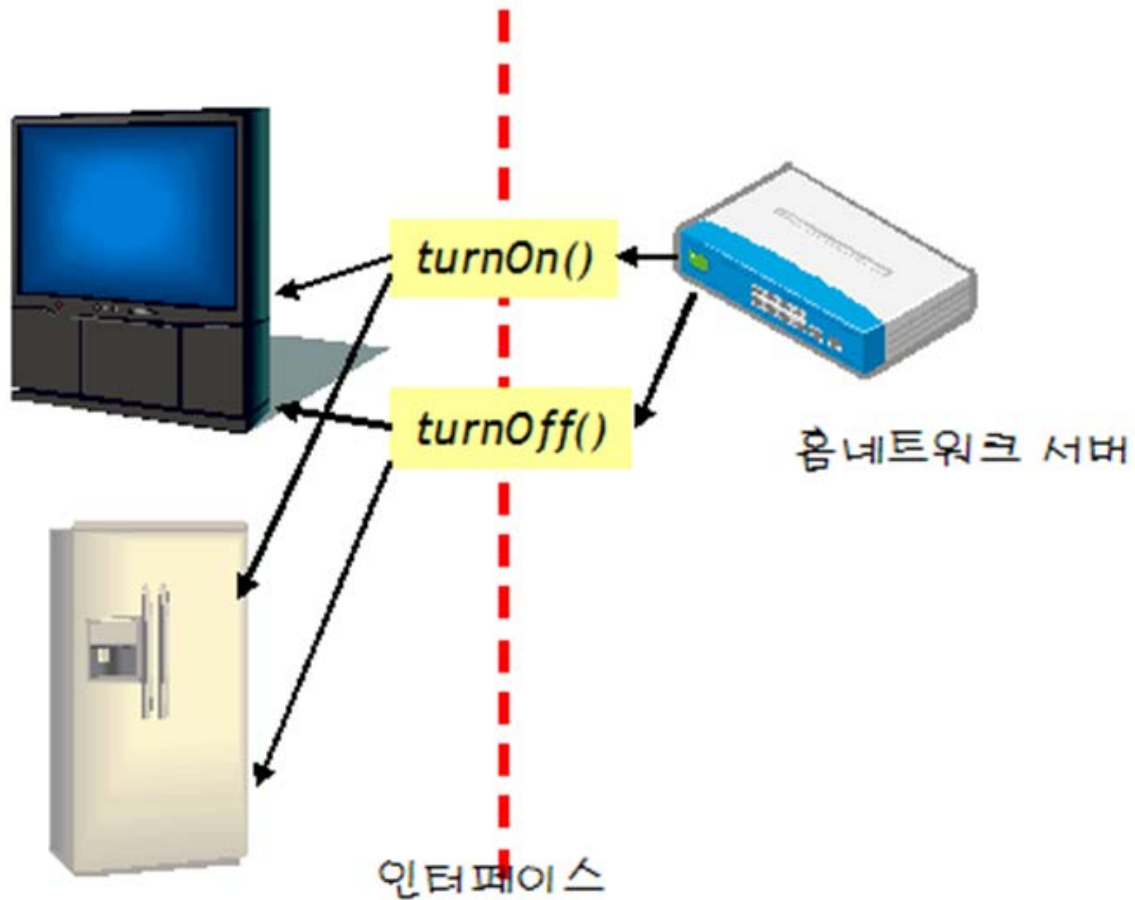
순수 가상 함수 (계속)

- 추상 클래스를 인터페이스로
 - 추상 클래스는 객체들 사이에 상호 작용하기 위한 인터페이스를 정의하는 용도로 사용할 수 있다.



순수 가상 함수 (계속)

- 인터페이스의 예



순수 가상 함수 (계속)

- 예제

```
class RemoteControl {  
    // 순수가상함수정의  
    virtual void turnON() = 0;    // 가전 제품을 켜다.  
    virtual void turnOFF() = 0;   // 가전 제품을 끄다.  
}  
  
class Television : public RemoteControl {  
    void turnON()  
    {  
        // 실제로 TV의 전원을 켜기 위한 코드가 들어간다.  
        ...  
    }  
    void turnOFF()  
    {  
        // 실제로 TV의 전원을 끄기 위한 코드가 들어간다.  
        ...  
    }  
}
```

순수 가상 함수 (계속)

- 예제 (계속)

```
int main()
{
    Television *pt = new Television();
    pt->turnOn();
    pt->turnOff();

    Refrigerator *pr = new Refrigerator();
    pr->turnOn();
    pr->turnOff();

    delete pt;
    delete pr;
    return 0;
}
```